For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

The 5-step design approach

(20-25 mins)

1. Problem Statement
2. Functional Requirements
3. Non-functional Requirements
4. Scale Estimation

(25-20 mins)

5. **System Design**

Problem Statement

Reason by analogy

Find existing companies / systems that offer a similar product / feature.
Gives you an overview of the umbrella of different scopes that you can consider.

- **Whatsapp**
  - Telegram / Signal / Hike
  - Mobile First
  - ***Realtime chat (ultra low latency)***
  - built as a SMS replacement
- **Slack**
  - MS Teams / Flock
  - B2B - built for enterprise chat
  - ***Group chat***
- **Facebook Messenger**
  - Instagram Messages
  - Messages for social media
  - ***1-1 messages***

What not to build

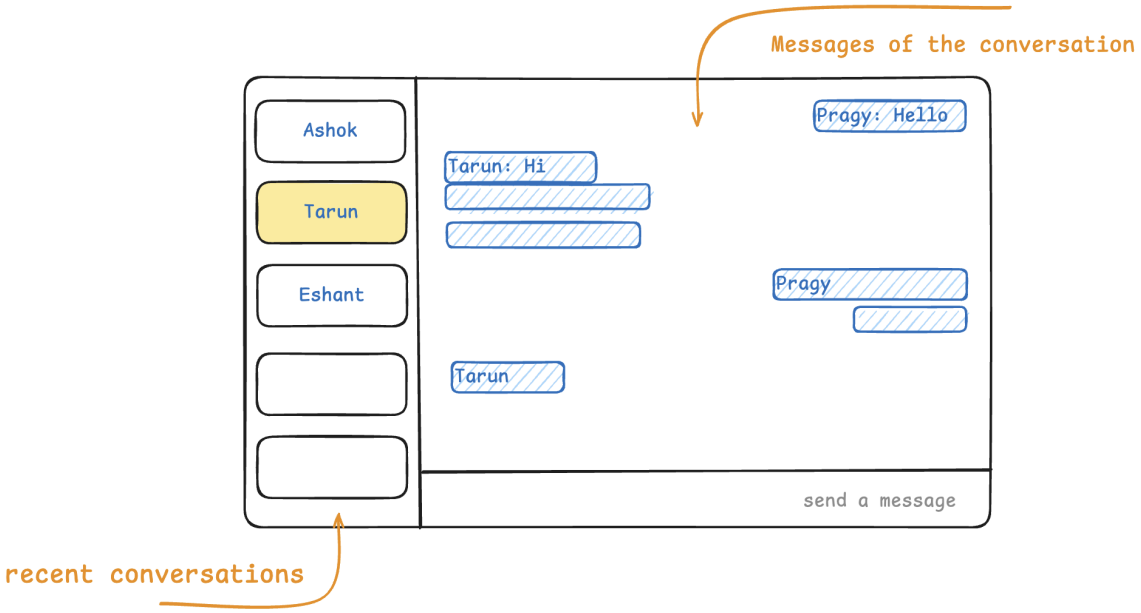
- Discord

    - Voice chat
- Orkut
    - Social Media
- Twitter
    - X / Threads
    - Social Media
- Skype
    - Video Chat & Screen Sharing
- Hangouts / Google Meet
    - Video Chat
- Snapchat
    - Privacy

Functional Requirements (FR)

MVP Features

Some feature/functionality that you offer to the client (user / another internal system).
You explicitly expose an API / write code for a functional requirement.



Actors

1. Users
2. ~~(enterprise software - Slack) Organisation~~

Whenever you think of features, think of what things the Actors can do in the app.

MVP Features

Always keep the "minimal" in your mind. MVP features is not a feature suggestion competition.

KISS: keep it simple, ~~silly~~.

~~Practice makes perfect!~~
Correct practice makes perfect. Incorrect practice harms you.

MVP features perfect for discussion (v0)	**Future Scope** good to have, but not absolutely necessary (v1+)	**Bad** get yourself rejected in an interview (v-never)

A user should be able to send messages to other users
 	
`sendMessage(sender_id: uuid,`
`recepient_id: uuid,`
`message: json ): ack/failure`
`user-facing`
	Users should be able to see the status of the messages they've sent
	• **Sent:** received by backend server
	• **Delivered:** delivered to the recipient's app, but hasn't been read yet
	• **Read:** the recipient has actually read the message
		Rich messages
		• react to the messages with Emojis
		• Videos / Images
		• Gifs

Users should be able to receive messages, & view the past messages in a conversation (most recent first).
Users should only be able to view messages of conversation that they're a part of.
 	
`getMessages(user_id: uuid,`
`conversation_id: uuid,`
`pagination_offset: int,`
`pagination_limit: int ): list of messages`
`user-facing`
 	
conversation_id: identifies the conversation (Tarun+Sanjana, or, Harini+Pallavi, ..)
	Updating/Deleting already sent messages (extremely hard to do correctly)	Profile management

Users should be able to view the conversations that they're a part of (most recent first)
 	
`getConversations(user_id: uuid,`
`pagination_offset: int,`
`pagination_limit: int ): list of conversation`
`({conversation_id, name, unread count, a quick summary of the latest message})`
`user-facing`
	Notifications	Broadcast same message to multiple people (batch processing)

Users can participate & send messages inside groups. A single group can potentially have 100,000+ users `sendMessageGroup(sender_id: uuid,`	Multi-device support (extremely hard to do correctly - CRDTs)	Group Management • creating • adding people • permissions

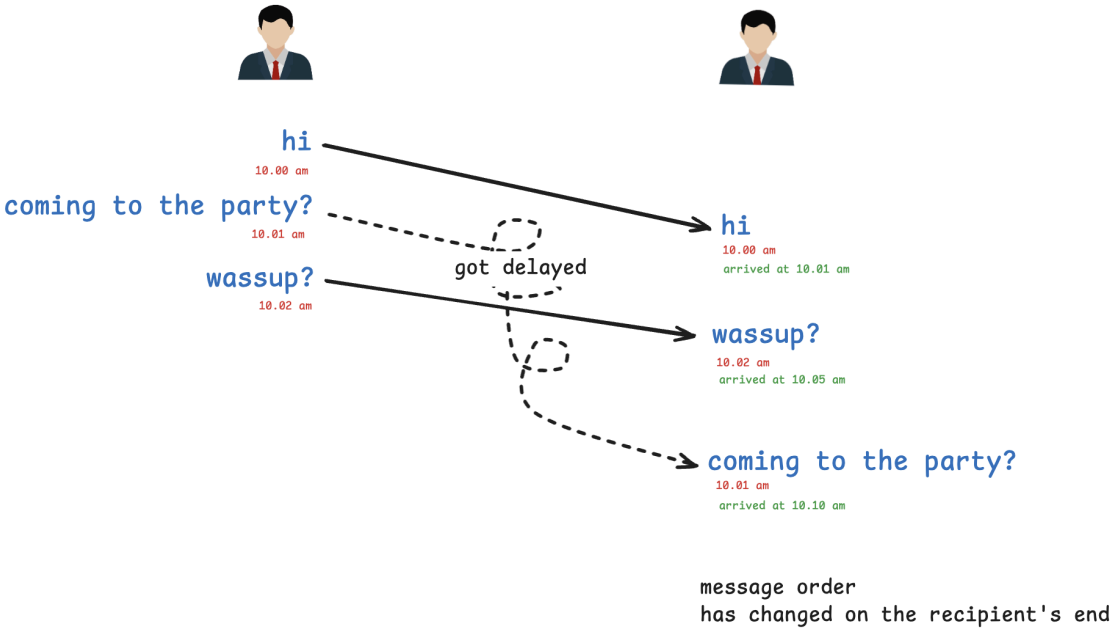
`group_id: uuid,`
`message: json  ): ack/failure`
user-facing
 	
`getMessagesGroup(user_id: uuid,`
`group_id: uuid,`
`pagination_offset: int,`
`pagination_limit: int  ): list of messages`
user-facing
		• group description

		App should suggest / load contacts from the user's phone
		Message Forwarding
		• User should be able to start a new conversation by selecting a recipient from list of contacts/friends • Create a new conversation • Delete conversation • Mark as Spam
		Syncing messages b/w the app cache & cloud

Non-Functional Requirements (NFR)

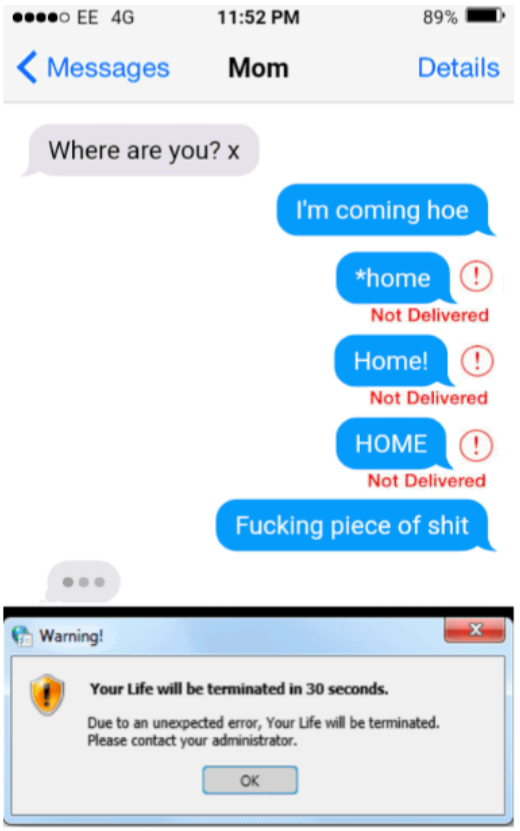
Design Goals

Message Order



Why is message order important?

Meaning of a conversation can change based on the message order.



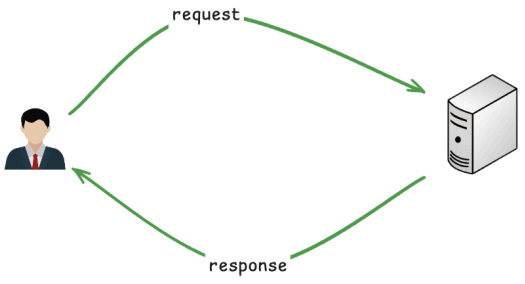
Idempotency

Messaging apps are typically mobile first. Mobile users don't always get perfect internet connections. Therefore, messaging apps have to frequently deal with n/w failures.

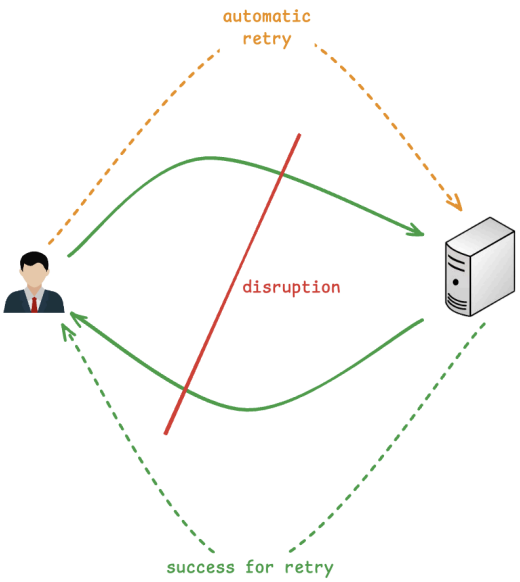
In a lot of scenarios, the frontend app (client-side) is coded to automatically retry sending messages/posts/comments if the sending fails — because we don't want the user to re-type the entire message.

```
<input id="message"
       type="text"
       placeholder="send a message"
       onkeydown="sendMessage" />
<script>
    function sendMessage() {
        message = document.getElementById("message");
        sendWithRetries(message);
    }

    function sendWithRetries(message) {
        fetch("/backend-api-url/messages", "POST", {
            message: message
        }).then(() => console.log("success"))
          .error(() => {
              // automatic retry after 1 second
              setTimeout(() => sendWithRetries(message),
                         1)
          })
    }
<script>
```



Typical Request/Response



In case of a failure

However, if

- the initially request successfully goes to the server
- the server acts on the request (stores the message/post/..)
- responds with an ACK
- but the acknowledgement gets lost and never reaches the user

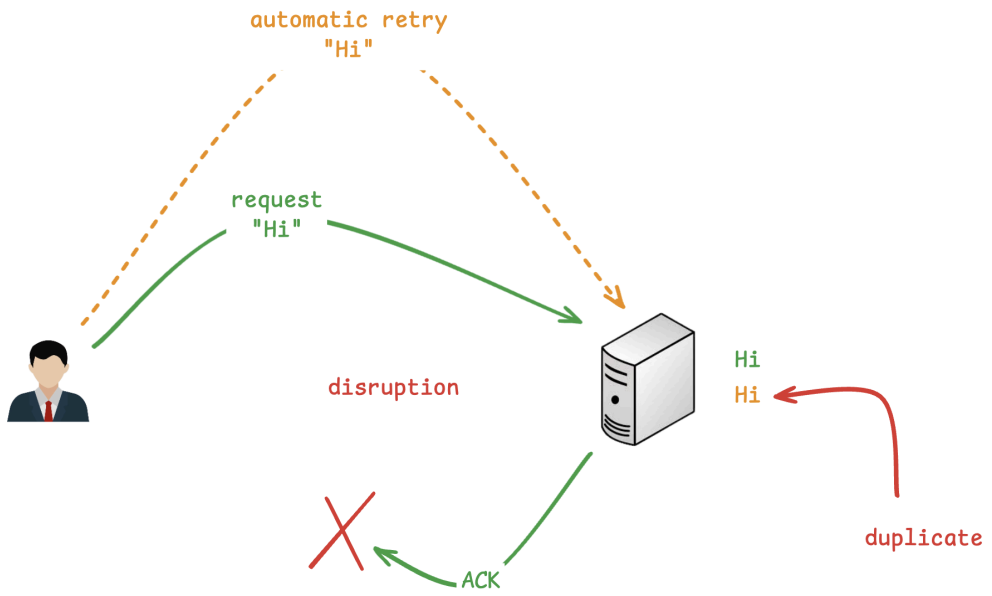
in this case,

- the frontend will assume (incorrectly) that the server didn't receive our request
- automatically retry

due to this

- the server will end up storing the message/post again
- there will be duplicates

Automatic retries on the frontend can cause duplicates in the database!



Idempotent

A function `f(x)` is said to be idempotent if & only if `f(x) = f(f(x))`
Repeated application of the same function does nothing more than a single application.

`f(x) = x`^{2} is NOT idempotent. Because `f(4) = 16`, but `f(f(4)) = 256`.
`f(x) = |x|` is idempotent. Because `f(-5) = 5` and `f(f(-5)) = 5`

Any POST api endpoints should be idempotent if your frontend can retry.

Q: Does the backend engineer know / care about whether the frontend guy used automatic retries?

No. They don't care.
It could be that currently the frontend doesn't retry, but later on, someone adds auto-retries to the frontend.

Therefore, irrespective of what the frontend does, we should always try to make our backend POST apis idempotent whenever possible.

Note that if the user wants to purposefully resend the same message again & again, they should be allowed to.
When I want to annoy my friends, I will send them "Hi" 20 times in a row. This should be allowed!
Duplicates **due to automatic retries** should be prevented.

Consistency vs ~~Availability~~

PACELC

Do NOT jump to an answer

Q: What is the data ?

The messages that are being sent

Q: What does eventual consistency (stale reads) mean in this context ?

- Messages are out of order on the recipient's side
- Can't see the latest message
    - Sender side
        - If Shaurya sends a message to Harpal at `t=0`
        - The server ACK'd Shaurya's message
        - When Shaurya reloaded his app at `t=1`, he is not able to see the message that he sent (because the read was stale)
        - **Shaurya thinks that his message got deleted by the app**
        - In reality the message is not deleted/lost, if Shaurya waits long enough, he will be able to see the message eventually
        - This type of thing is typically solved by providing "read-your-write" consistency.
    - Receiver side
        - If Shaurya sends a message to Harpal at `t=0`
        - The server ACK'd Shaurya's message
        - At `t=1` when Harpal (recipient) tries to load his messages, he isn't able to see the message that Shaurya sent.
            - Shaurya says that he's the message
            - **But Harpal cannot receive the message**
            - In reality the message is not deleted/lost, if Shaurya waits long enough, he will be able to see the message eventually

Q: Can we afford stale reads in this situation?

No, we cannot afford to have such stale reads.

Q: What does data loss mean in this context ?

Sender sent a message, and the server ACK'd
but, the message actually got lost by the backend.

Q: Can we afford data loss in this situation?

Very very bad. No.

Communication is the backbone of Human Society
Communication needs strong consistency!

Consistency vs Latency

What are our latency requirements?

Low Latency: < 5 seconds

We need realtime chat: if PersonA has sent a message to PersonB, and PersonB is online, then PersonB should receive the message within a few seconds.

But availability is also important!

But the PACELC theorem says that we can't have both Consistency & Latency!
Is it possible to achieve high consistency along with high availability & low latency?
No - PACELC says its impossible.

However, we can create the illusion of consistency with availability & low latency

Scale Estimation

The scale will dictate our design choices

assumption: Total users = *(web/planet scale)* = `2 billion`

Daily Active Users (DAU): `80% of all users = 1.6 billion ~= 2 billion`
Typically, we consider the Pareto Principle & say that only 20% of the users will be active.
Will this be true for messaging apps like Whatsapp? **No!**
Instead of only 20%, for messaging apps like whatsapp, the number will be much higher
Note, for FB messenger, it might still be as low as 20%

assumption: Avg. number of messages per active-user per day
`= 10 to 20 messages / active-user / day`

Total number of messages / day
`= (20 messages / active-user / day) * (2 billion active-users)`
`= 40 billion messages / day`

Avg. messages / second

= 40 billion messages / day
= $4 * 10^{10}$ messages / (10^5 seconds)
= $4 * 10^5$ messages / second
= 400,000 messages / second

- **`sendMessage(...)`:** `400,000 per second avg`
- **`getMessages(...)`:**
    - a message is sent once, but read multiple times
    - but not by a huge margin
        - in Stackoverflow/Twitter, a post is made once, and read thousands or millions of times
        - in Whatsapp, a message is sent once, but read a handful of times
    - **assumption:** `= 2x of sendMessage`
      `= 800,000 per second`
    - This statistic will change for groups
        - in a large group, a message is sent once, but read thousands of times
        - *if we have `400,000` messages/sec and each message is read `100,000` times then we have `40 billion requests/sec` — this number is insane - no system (not even Google with its 10 million servers) can handle this as of 2025.*
        - however, in groups, the number of users participating across the globe is much lower
            - slack users <<< whatsapp users
            - slack only has `42 million` DAU compared to whatsapp's `2 billion`.
- **`getConversations(...)`:** only when the user re-opens the app

Peak Load

assumption: `= 5x of the avg load` during global events like Covid or New Year
`= 2 million messages / second`

Total amount of data (over 20 years)

What's the data?

```
message: {
    sender_id: uuid (16b)
    receiver_id/group_id: uuid (16b)
    message_id: uuid (16b)
    text: string (200b avg)
    when: timestamp  (8b)
    where: geolocation (16b)
    ~~delivery_status:~~
    attachment_url: string (200b avg)
}
```

Avg. size `= 500 bytes/message`

Amount of data / day
`= (500 bytes / message) * (40 billion messages / day)`

```
= 20 trillion bytes / day
= 20 TB / day
```

Amount of data over 20 years

```
= (20 TB / day) * (20 years)
= (20 TB / day) * (20 * 365 days)
= (20 TB / day) * (~ 10,000 days)
= 200 Petabytes
```

Can this amount of data fit on a single server?

No. As of 2025, we can store up to 2 PB on a single server, not 200PB. Even if we could, a single server won't be able to handle 2 million messages/sec.

We definitely need sharding!

Read Heavy / Write Heavy?

Writes (sendMessage): 400,000 / second
Reads (getMessages): 800,000 / second

Neither is 10x or more larger than the other - neither is dominating.
This is both read & write heavy!

It is very challenging to design systems which are both read & write heavy.

This means that we ill either have to convert this into a read heavy system or into a write heavy system.
Can we reduce the writes (batching/sampling)? No.
So the only solution is to reduce the reads that go the database — by absorbing the reads in the cache.

So we will have lots of cache, and the database will optimized for writes.

System Design

Idempotency & Message Order

Each message will have a **unique message id that is generated on the client side**

1. *UUID / ULID (see in HLD - 16.1 - Unique ID Generator)*

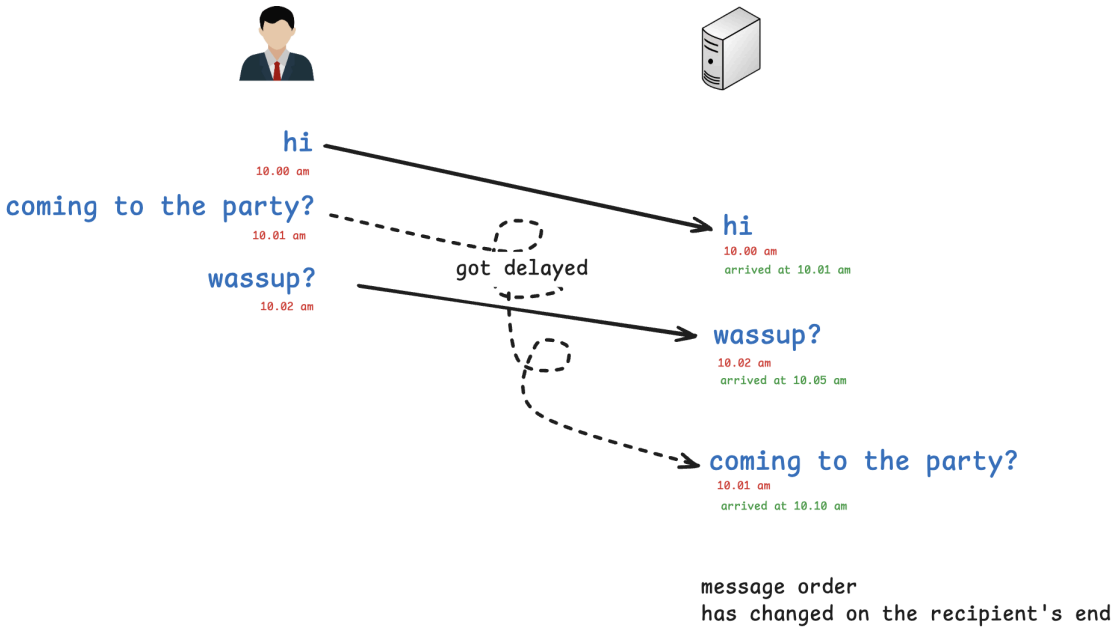
Idempotency

- whenever the backend receives a `sendMessage` API call, it stores the message in the DB only if the `message_id` is not already there in the DB
- if the `message_id` is already there in the DB, it will just ignore the request, but, it will still send a success response (so the frontend can stop retrying)

```
<input id="message"
       type="text"
       placeholder="send a message"
       onkeydown="sendMessage" />
<script>
     function sendMessage() {
          var content = document.getElementById("message");
          var message = {
              id: uuid_v4(), // note that this id will
                             // remain unchanged for the retries
              content: content,
              timestamp: new Date(),
          }
          sendWithRetries(message);
      }
      function sendWithRetries(message) {
          fetch("/backend-api-url/messages", "POST", {
              message: message
          }).then(() => console.log("success"))
            .error(() => {
                // automatic retry after 1 second
                setTimeout(() => sendWithRetries(message),
                           1)
          })
      }
<script>
```

Message Order

1. Sorting based on `created_time` timestamp will NOTwork
    - because the recipient doesn't even know that there's a message pending until the message arrives
    - by the time the receiver's app realises that an intermediate message arrived late, it was already too late - it had already shown the later messages to the user
2. note that no backend based solution (SQS/buffer/...) will work
    - because the backend doesn't even know that there's a message pending until the message arrives
    - by the time the server realises that an intermediate message arrived late, it was already too late - it had already put later messages in the queue

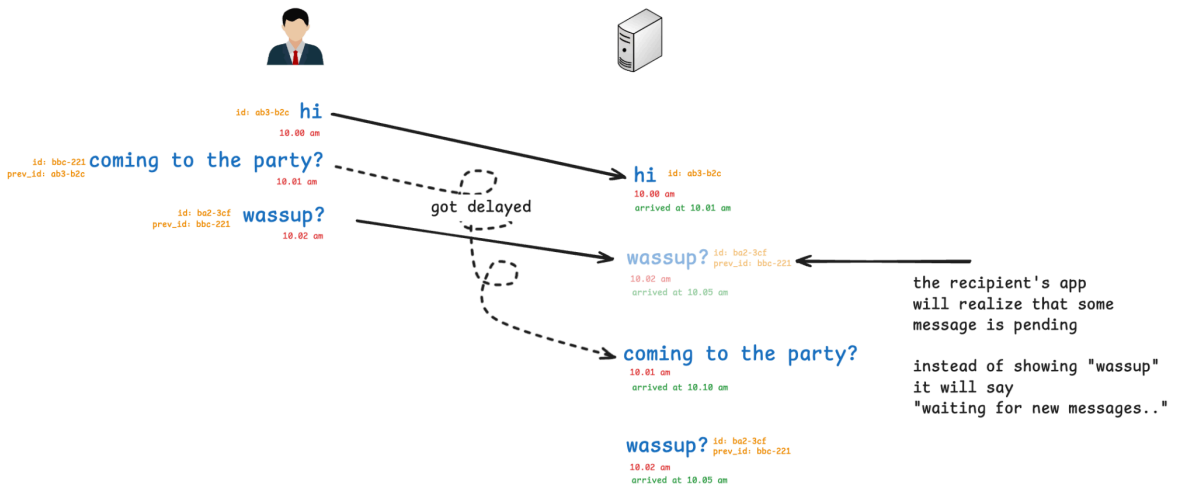


Solution: create a message chain

Every new message should contain the `previous_message_id` inside it.

On the recipient's side, the app will not show a message unless its previous message has also been received & shown.

Instead, if a message appears out-of-order, it will show `waiting-for-messages` until all the previous messages have arrived.

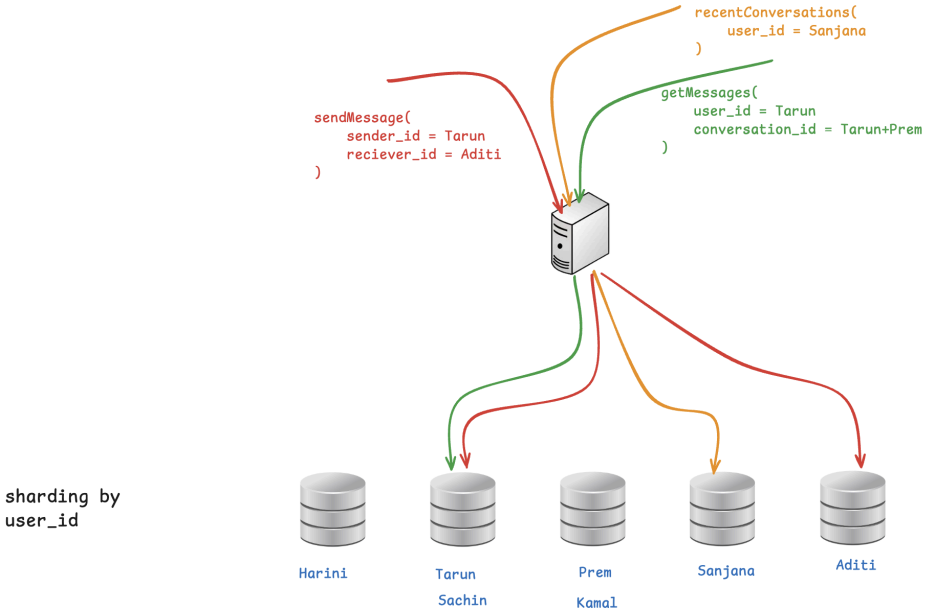



```
<input id="message"
       type="text"
       placeholder="send a message"
       onkeydown="sendMessage" />
<script>
     var previous_id = null;
     function sendMessage() {
```

```
            var content = document.getElementById("message");
            var id = uuid_v4()
            var message = {
                previous_id,     // id of last message that I sent
                id: id,
                content: content,
                timestamp: new Date(),
            }
            previous_id = id;
            sendWithRetries(message);
        }
    <script>
```

Sharding

Sharding Key: `user_id`



1-1 messages

getMessages(user_id, conversation_id, ...)

Only go to the user's shard, because all messages sent/received by the user will be present in their shard.

sendMessage(sender_id, receiver_id, ...)

Go to both sender & receiver's shard (data replication)

recentConversations(user_id)

Only go to the user's shard

~~Group Messages~~

getMessages(user_id, group_id, ...)

1. **Store in Sender Only:** Reads will be fan-out
   because messages sent by different senders in the same group will be stored in different shard
2. **Store in All Participants:** fast reads, because we can read from any shard

sendMessage(sender_id, group_id, ...)

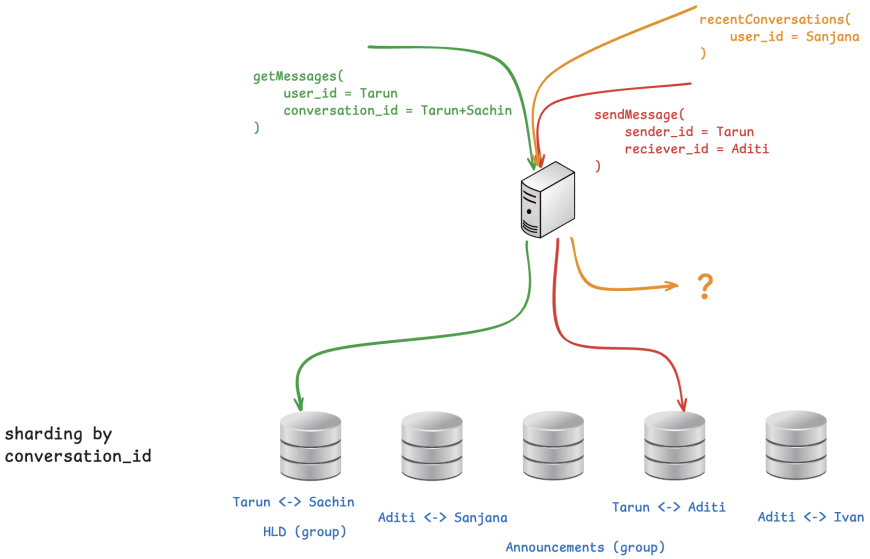
1. **Store in Sender Only:** Store it only in the sender's shard
2. **Store in All Participants' shards:** Fan-out writes

recentConversations(...)

Basically, sharding by user_id will NOT work for group conversations.
Either reads will be fan-out, or writes will be fan-out.

Sharding Key: `conversation_id`

For groups, conversation_id = group_id
For 1-1 chats, conversation_id = unique id assigned to a pair of users



1-1 messages

getMessages(user_id, conversation_id, ...)

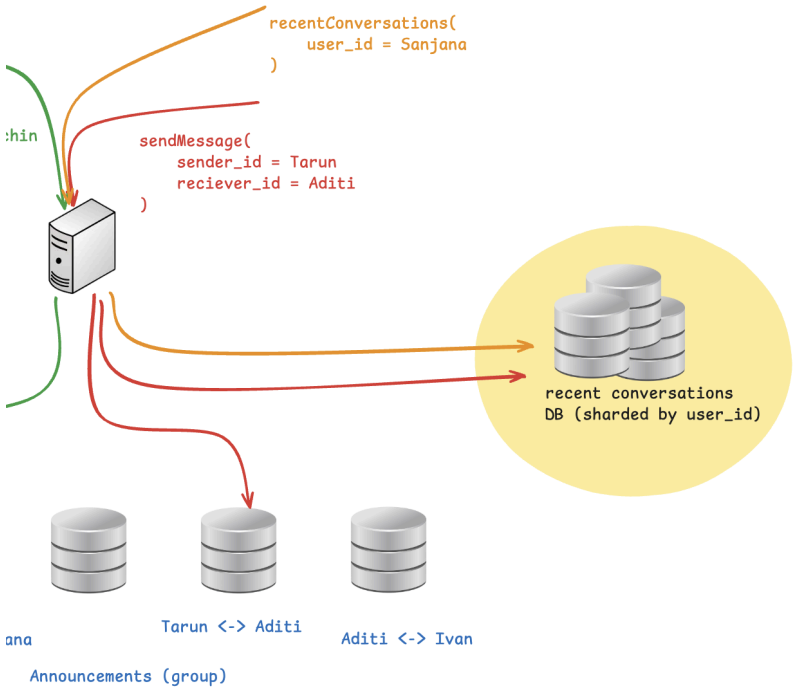
Only go to the conversation's shard, because all messages sent/received by the any participant of that conversation will be present in that shard.

sendMessage(sender_id, receiver_id, ...)

Go to conversation's shard

recentConversations(user_id)

~~Will be fan-out => bad~~
We can have a separate db to store the most recent conversations.
Note that now, the sendMessage must also update the recentConversations DB.



Note: the recentConversations DB holds information (last message timestamp) about all conversations.
It doesn't only store recentConversations.
It's called the recentConversations DB because we're using it to answer the recentConversations(...) query exclusively.

Group Messages

Note that a group is just a conversation amongst a lot of people.

getMessages(user_id, group_id, ...)

just go to the group's shard

sendMessage(sender_id, group_id, ...)

just store in the group's shard

~~**recentConversations**(user_id)~~

No matter what you do, the moment a message is sent in a group, the most recent conversation gets updated for potentially 100,000 users!
This will always be a fan-out no matter what you do.

Therefore, you will not provision this API for group conversations.
Practically: happens only at frontend.
Client side app can maintain a cache of recent conversations.
This cache data is updated thanks to notifications.

What to choose?

- **Facebook Messenger / Whatsapp:** Group size is limited - sharding by `user_id` is ideal
- **Slack / MS Teams:** Groups are a core feature, and groups can potentially get very large (100,000+ users) - sharding by `conversation_id` is ideal

Consistency

PACELC: Consistency vs Availability/Latency

PACELC says if we want consistency, we have to give up on availability & latency.
We do want consistency.

Assume that we're sharding by user_id. In this case, any write must go to 2 shards - sender & receiver.
And we must maintain consistency during this write.

Option 1: use 2 Phase Commit (2PC) to write to both shards atomically
- extremely slow (low throughput & high latency)
- low availability

Can we instead create an illusion of consistency?
Instead of trying to write atomically, let's try to write 1 by 1.

~~Write to Sender shard first~~

- **Write the message to Sender's shard**
  - Failure
    - immediately return a failure
    - client can retry to send the message
  - Success
    - **Write to the Receiver's shard**
      - Success
        - immediately return a success
          (we've successfully written to both sender & receiver's shard)
      - Failure
        - the data is in an inconsistent state (written to sender shard, but not to receiver shard)
        - right now, the sender thinks that the message has been sent, but if the recipient tries to load their messages, they don't see this message
        - Retry
          - the sender has to wait until the retry succeeds
          - very high latency, because the sender has to wait until the retry finally succeeds
        - ~~Rollback~~
          - we can return an error
          - but now, the system is in an inconsistent state until the rollback completes.

Write to Receiver shard first

- **Write to the Recipient's shard**
  - Failure
    - Immediately return a failure
    - Client can retry
  - Success
    - the message has actually been sent!
      At this point, if the recipient checks their messages, they will be able to see this new message
    - **Immediately return a success**
      (without even writing to sender shard)
      very low latency
    - **(async) Write to the Sender's shard**
      - Success
        - great, we're done
      - Fails
        - just keep retrying until it succeeds

- the sender isn't waiting, they're already received an acknowledgement

Note that if the initial write to Recipient's shard succeeds, but the initial write to Sender's shard fails - when the sender tries to reload their app, they will not be able to see the message that they just sent.
This means that the sender thinks that their message has been deleted => bad!

We can fix this by simply having a frontend cache (cache inside the client app)
The app will know what messages have been sent. Even though the getMessages API doesn't show this message, the frontend app will know that this message was sent and acknowledged by the server.

The only situation when this will be an issue will be when the sender sends a message, the sender's shard is misbehaving in the backend, the server reinstalls their frontend app (or clear the app cache), and then they try to see the message.
This is v.v. rare, so not an issue.

By cleverly writing to recipient's shard first, and utilizing the frontend cache at the sender side, we can create an illusion of immediate consistency.
It solves the practical problem for us.

Consistency vs Availability

Imagine that there's a partition.
`Harini  ----- "Hi" -----> Aditi`
App Server 1 receives this request.
App server 1 is able to communicate with Aditi's Shard, but because of a n/w partition, it is not able to communicate with Harini's Shard.

App server will just write to Aditi's shard (recipient), and return success to Harini.
It will also drop a message in a task queue to add this message to Harini's shard in an async manner.

Even in case of n/w partition, the sendMessages & getMessages remains available => **High Availability**

Aditi can see the message because it is written to her shard. And because of the frontend cache, Harini can also see the message. => **(illusion) Immediate Consistency**

Note: if the write to Aditi's shard fails, then we can return an error to Harini & then they can retry.

Consistency vs Latency

Because we're not actually trying to write to both shards in an atomic manner, we don't have to wait for retry/rollback to complete.
We can just return success the moment we write to recipient shard.
Low Latency

Choosing the right Database

Writes (sendMessage): 200,000 / second
Reads (getMessages): 400,000 / second
Both Read & Write Heavy!

No database is optimized for both reads & writes!

Optimizing Reads/Writes

~~Can we optimize writes?~~

One can optimize writes by either batching or sampling

- **Batching:** causes stale reads
  but we want low latency
- **Sampling:** causes potential data loss
  we cannot afford data loss

Can we optimize reads?

Just try to absorb as many reads as possible via a cache.

Now that we're absorbing majority of reads in the cache, we can optimize our database for writes!

Ideal Database

Requirements

1. High write throughput (200,000 writes/sec avg)
2. No need of joins
3. No need of search (if we need it later, we can provision another database in a separate microservice)
4. Paginated reads (by timestamp)
5. No need of transactions
6. We want disk persistence

Ideal database is therefore a **Wide Column Database - HBase**.

Cache

Local vs Global (Single) **vs Global** (Distributed)

- Do we need to transfer lots of data b/w app server & cache server to handle a read request?
  `(getMessages(user_id, conversation_id, pagination))`
  No.
- We should probably think of a Global Cache
  - Will a single cache server be able to handle all the data?
    No.
  - We're looking at a distributed global cache
    - But in this case, what is the app server for the `getMessages` API doing?
      Nothing - just passing the request forward to the cache server

So instead of having a global cache and then having useless app servers, we can **simply use a local cache**.
App servers now double as the cache.

Invalidation

We want immediate consistency, but we don't want the latency that comes with immediate consistency.

- If we have a separate app server & cache server, immediate consistency requires 2 phase commit, because we've to keep the cache & db up to date (2 places)
- If we have local cache, then writing to the cache is always successful. No need of 2PC here.

Because of the local cache, we can get write-through invalidation (immediate consistency) without high latency.

Eviction

LRU eviction

Routing

Our app servers are stateful. If an app servers holds Harini's messages in cache, we want Harini's requests to always go to that app server.

Consistent Hashing.

What happens if a cache server crashes?

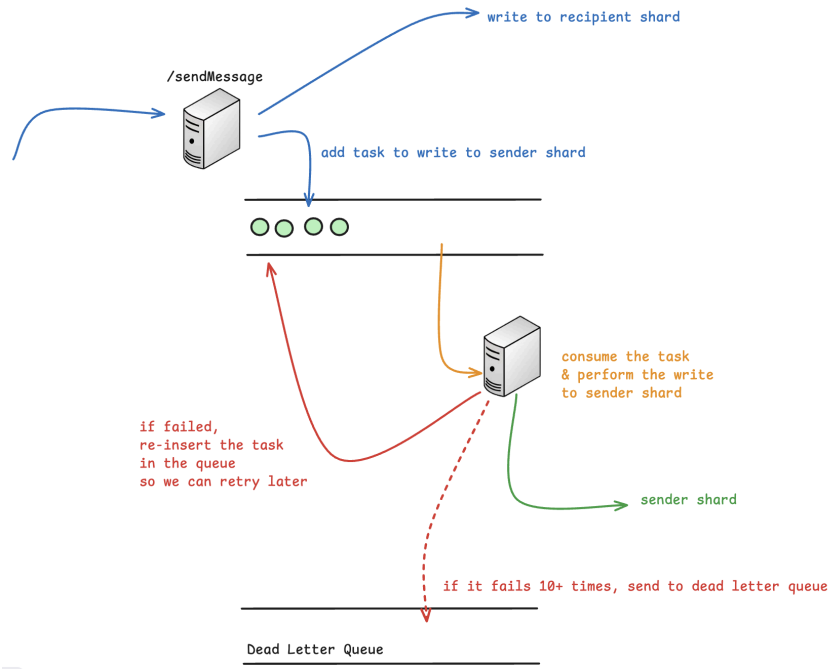
The users originally going to this server have to be redirected to another server. This other server doesn't have their messages cached.
Cold cache problem - data not in cache, fetch it from DB

Some of the time (when the cache server crashes), some of the users (only those that were originally going to the now crashed server), will see a larger latency for reads. That is perfectly okay.

Doubts

If a message write to sender shard fails multiple times back-to-back, then this message will be sent to a "Dead Letter Queue"

Dead Letter Queue: a special topic/category inside your message queue where messages get placed if they fail multiple times in a row.
Something has gone horribly wrong, more retries will not solve this, so stop retrying, and wait for a developer to intervene and figure it out.



Q: What is soft delete vs hard delete?

1. Consider that you're building a banking solution. If a user deletes their account, you cannot just delete all the transaction information – the govt needs you to store the history of transactions for at least 2 years (fraud detection, national security)
2. Companies also don't want to lose your data. Netflix uses data about what users watch what videos to train their recommender system. Now just because you're deleted your account, it doesn't mean that they want to lose our on this juicy training data.

Hard Delete
users
id, name, password_hash

delete from users where id = 123

select * from users

Soft Delete
users
id, name, password_hash, is_deleted

update users set is_deleted = true where id = 123

select * from users where is_deleted = false